User:

You will be helping me implement my deep learning homework. I will copy paste code that you need to fill in. Do not make any assumptions, and ask in you are unsure about anything.

Claude:

I'd be happy to help you with your deep learning homework! Please go ahead and paste the code you need help filling in.
I'll ask clarifying questions if I'm unsure about:

- The specific framework/library being used (PyTorch, TensorFlow, JAX, etc.)
- Expected input/output shapes or types
- Specific algorithms or methods you're expected to implement
- Any constraints or requirements from your assignment instructions

Feel free to share the code whenever you're ready!

User:

fill in the TODO to implement the reparameterization trick for sample_gaussian.

```
import numpy as np
import os
import shutil
import torch
from models.vae import VAE
from torch.nn import functional as F
from torchvision import datasets, transforms

bce = torch.nn.BCEWithLogitsLoss(reduction='none')

############################################################################
####
# Please familiarize yourself with the code below.
#
# Note that the notation is
# argument: argument_type: argument_shape
#
# Furthermore, the expected argument_shape is only a guideline. You're free to
# pass in inputs that violate the expected argument_shape provided you know
# what you're doing
############################################################################
####

def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian
```

```
    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """

################################################################################
####
    # TODO: Modify/complete the code here
    # Sample z

################################################################################
####


################################################################################
####
    # End of code modification

################################################################################
####


def gaussian_parameters(h, dim=-1):
    """
    Converts generic real-valued representations into mean and variance
    parameters of a Gaussian distribution

    Args:
        h: tensor: (batch, ..., dim, ...): Arbitrary tensor
        dim: int: (): Dimension along which to split the tensor for mean and
            variance

    Returns:
        m: tensor: (batch, ..., dim / 2, ...): Mean
        v: tensor: (batch, ..., dim / 2, ...): Variance
    """
    m, h = torch.split(h, h.size(dim) // 2, dim=dim)
    v = F.softplus(h) + 1e-8
    return m, v
```

```
def log_bernoulli_with_logits(x, logits):
    """
    Computes the log probability of a Bernoulli given its logits

    Args:
        x: tensor: (batch, dim): Observation
        logits: tensor: (batch, dim): Bernoulli logits

    Return:
        log_prob: tensor: (batch,): log probability of each sample
    """
    log_prob = -bce(input=logits, target=x).sum(-1)
    return log_prob


def kl_cat(q, log_q, log_p):
    """
    Computes the KL divergence between two categorical distributions

    Args:
        q: tensor: (batch, dim): Categorical distribution parameters
        log_q: tensor: (batch, dim): Log of q
        log_p: tensor: (batch, dim): Log of p

    Return:
        kl: tensor: (batch,) kl between each sample
    """
    element_wise = (q * (log_q - log_p))
    kl = element_wise.sum(-1)
    return kl


def kl_normal(qm, qv, pm, pv):
    """
    Computes the elem-wise KL divergence between two normal distributions KL(q || p) and
    sum over the last dimension

    Args:
        qm: tensor: (batch, dim): q mean
        qv: tensor: (batch, dim): q variance
        pm: tensor: (batch, dim): p mean
        pv: tensor: (batch, dim): p variance
```

```
    Return:
        kl: tensor: (batch,): kl between each sample
    """
    element_wise = 0.5 * (torch.log(pv) - torch.log(qv) + qv / pv + (qm - pm).pow(2) / pv - 1)
    kl = element_wise.sum(-1)
    return kl


def duplicate(x, rep):
    """
    Duplicates x along dim=0

    Args:
        x: tensor: (batch, ...): Arbitrary tensor
        rep: int: (): Number of replicates. Setting rep=1 returns orignal x

    Returns:
        _: tensor: (batch * rep, ...): Arbitrary replicated tensor
    """
    return x.expand(rep, *x.shape).reshape(-1, *x.shape[1:])


def log_mean_exp(x, dim):
    """
    Compute the log(mean(exp(x), dim)) in a numerically stable manner

    Args:
        x: tensor: (...): Arbitrary tensor
        dim: int: (): Dimension along which mean is computed

    Return:
        _: tensor: (...): log(mean(exp(x), dim))
    """
    return log_sum_exp(x, dim) - np.log(x.size(dim))


def log_sum_exp(x, dim=0):
    """
    Compute the log(sum(exp(x), dim)) in a numerically stable manner

    Args:
        x: tensor: (...): Arbitrary tensor
        dim: int: (): Dimension along which sum is computed
```

```
    Return:
        _: tensor: (...): log(sum(exp(x), dim))
    """
    max_x = torch.max(x, dim)[0]
    new_x = x - max_x.unsqueeze(dim).expand_as(x)
    return max_x + (new_x.exp().sum(dim)).log()


def load_model_by_name(model, global_step):
    """
    Load a model based on its name model.name and the checkpoint iteration step

    Args:
        model: Model: (): A model
        global_step: int: (): Checkpoint iteration
    """
    file_path = os.path.join('checkpoints',
                             model.name,
                             'model-{:05d}.pt'.format(global_step))
    state = torch.load(file_path)
    model.load_state_dict(state)
    print("Loaded from {}".format(file_path))


################################################################################
####
# No need to read/understand code beyond this point. Unless you want to.
# But do you tho ¯\_(ツ)_/¯
################################################################################
####


def evaluate_lower_bound(model, labeled_test_subset, run_iwae=False):
    check_model = isinstance(model, VAE) or isinstance(model, GMVAE)
    assert check_model, "This function is only intended for VAE and GMVAE"

    print('*' * 80)
    print("LOG-LIKELIHOOD LOWER BOUNDS ON TEST SUBSET")
    print('*' * 80)

    xl, _ = labeled_test_subset
    torch.manual_seed(0)
    xl = torch.bernoulli(xl)
```

```
    def detach_torch_tuple(args):
        return (v.detach() for v in args)

    def compute_metrics(fn, repeat):
        metrics = [0, 0, 0]
        for _ in range(repeat):
            niwae, kl, rec = detach_torch_tuple(fn(xl))
            metrics[0] += niwae / repeat
            metrics[1] += kl / repeat
            metrics[2] += rec / repeat
        return metrics

    # Run multiple times to get low-var estimate
    nelbo, kl, rec = compute_metrics(model.negative_elbo_bound, 100)
    print("NELBO: {}. KL: {}. Rec: {}".format(nelbo, kl, rec))

    if run_iwae:
        for iw in [1, 10, 100, 1000]:
            repeat = max(100 // iw, 1) # Do at least 100 iterations
            fn = lambda x: model.negative_iwae_bound(x, iw)
            niwae, kl, rec = compute_metrics(fn, repeat)
            print("Negative IWAE-{}: {}".format(iw, niwae))


def evaluate_classifier(model, test_set):
    check_model = isinstance(model, SSVAE)
    assert check_model, "This function is only intended for SSVAE"

    print('*' * 80)
    print("CLASSIFICATION EVALUATION ON ENTIRE TEST SET")
    print('*' * 80)

    X, y = test_set
    pred = model.cls.classify(X)
    accuracy = (pred.argmax(1) == y).float().mean()
    print("Test set classification accuracy: {}".format(accuracy))


def save_model_by_name(model, global_step):
    save_dir = os.path.join('checkpoints', model.name)
    if not os.path.exists(save_dir):
        os.makedirs(save_dir)
    file_path = os.path.join(save_dir, 'model-{:05d}.pt'.format(global_step))
    state = model.state_dict()
```

```
    torch.save(state, file_path)
    print('Saved to {}'.format(file_path))



def delete_existing(path):
    if os.path.exists(path):
        print("Deleting existing path: {}".format(path))
        shutil.rmtree(path)


def reset_weights(m):
    try:
        m.reset_parameters()
    except AttributeError:
        pass

def get_mnist_data(device, use_test_subset=True):
    preprocess = transforms.ToTensor()
    train_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=True, download=True, transform=preprocess),
        batch_size=100,
        shuffle=True)
    test_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=False, download=True, transform=preprocess),
        batch_size=100,
        shuffle=True)

    # Create pre-processed training and test sets
    X_train = train_loader.dataset.train_data.to(device).reshape(-1, 784).float() / 255
    y_train = train_loader.dataset.train_labels.to(device)
    X_test = test_loader.dataset.test_data.to(device).reshape(-1, 784).float() / 255
    y_test = test_loader.dataset.test_labels.to(device)

    # Create supervised subset (deterministically chosen)
    # This subset will serve dual purpose of log-likelihood evaluation and
    # semi-supervised learning. Pretty hacky. Don't judge :<
    X = X_test if use_test_subset else X_train
    y = y_test if use_test_subset else y_train

    xl, yl = [], []
    for i in range(10):
        idx = y == i
        idx_choice = get_mnist_index(i, test=use_test_subset)
```

```
            xl += [X[idx][idx_choice]]
            yl += [y[idx][idx_choice]]
        xl = torch.cat(xl).to(device)
        yl = torch.cat(yl).to(device)
        yl = yl.new(np.eye(10)[yl.cpu()])
        labeled_subset = (xl, yl)

        return train_loader, labeled_subset, (X_test, y_test)


def get_mnist_index(i, test=True):
    # Obviously *hand*-coded
    train_idx = np.array([[2732,2607,1653,3264,4931,4859,5827,1033,4373,5874],
                          [5924,3468,6458,705,2599,2135,2222,2897,1701,537],
                          [2893,2163,5072,4851,2046,1871,2496,99,2008,755],
                          [797,659,3219,423,3337,2745,4735,544,714,2292],
                          [151,2723,3531,2930,1207,802,2176,2176,1956,3622],
                          [3560,756,4369,4484,1641,3114,4984,4353,4071,4009],
                          [2105,3942,3191,430,4187,2446,2659,1589,2956,2681],
                          [4180,2251,4420,4870,1071,4735,6132,5251,5068,1204],
                          [3918,1167,1684,3299,2767,2957,4469,560,5425,1605],
                          [5795,1472,3678,256,3762,5412,1954,816,2435,1634]])

    test_idx = np.array([[684,559,629,192,835,763,707,359,9,723],
                         [277,599,1094,600,314,705,551,87,174,849],
                         [537,845,72,777,115,976,755,448,850,99],
                         [984,177,755,797,659,147,910,423,288,961],
                         [265,697,639,544,543,714,244,151,675,510],
                         [459,882,183,28,802,128,128,53,550,488],
                         [756,273,335,388,617,42,442,543,888,257],
                         [57,291,779,430,91,398,611,908,633,84],
                         [203,324,774,964,47,639,131,972,868,180],
                         [1000,846,143,660,227,954,791,719,909,373]])

    if test:
        return test_idx[i]

    else:
        return train_idx[i]

def prepare_writer(model_name, overwrite_existing=False):
    return None

def log_summaries(writer, summaries, global_step):
```

```
    Pass
```

Claude:

I'll implement the reparameterization trick for sample_gaussian.
The reparameterization trick allows us to sample from a Gaussian distribution in a way that's differentiable with respect to the mean and variance parameters. The key idea is:
$z = m + \sqrt{v} * \varepsilon$, where $\varepsilon \sim N(0, I)$
This way, the randomness is separated from the parameters (m, v), making backpropagation possible.
Here's the implementation:
python

```
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """

################################################################################
####
    # TODO: Modify/complete the code here
    # Sample z

################################################################################
####
    # Sample epsilon from standard normal distribution
    epsilon = torch.randn_like(m)
    # Reparameterization trick: z = mean + sqrt(variance) * epsilon
    z = m + torch.sqrt(v) * epsilon
    return z

################################################################################
####
    # End of code modification

################################################################################
####
```

Explanation:

1. torch.randn_like(m) - samples from N(0, I) with the same shape and device as m
2. torch.sqrt(v) - computes the standard deviation from variance
3. m + torch.sqrt(v) * epsilon - transforms the standard normal sample to have mean m and variance v

This allows gradients to flow through m and v during backpropagation, which is essential for training VAEs.

User:

implement negative_elbo_bound

```
import torch
import utils as ut
from models import nns
from torch import nn
from torch.nn import functional as F

class VAE(nn.Module):
    def __init__(self, nn='v1', name='vae', z_dim=2):
        super().__init__()
        self.name = name
        self.z_dim = z_dim
        # Small note: unfortunate name clash with torch.nn
        # nn here refers to the specific architecture file found in
        # codebase/models/nns/*.py
        # models = getattr(nns, nn)
        self.enc = nns.Encoder(self.z_dim)
        self.dec = nns.Decoder(self.z_dim)

        # Set prior as fixed parameter attached to Module
        self.z_prior_m = torch.nn.Parameter(torch.zeros(1), requires_grad=False)
        self.z_prior_v = torch.nn.Parameter(torch.ones(1), requires_grad=False)
        self.z_prior = (self.z_prior_m, self.z_prior_v)

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
```

```
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)


        logits = self.dec.decode(z)
        # compute metrics

################################################################################
####
        # TODO: Modify/complete the code here
        # Compute negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs should all be scalar

################################################################################
####


################################################################################
####
        # End of code modification

################################################################################
####
        return nelbo, kl, rec

    def loss(self, x):
        nelbo, kl, rec = self.negative_elbo_bound(x)
        loss = nelbo

        summaries = dict((
            ('train/loss', nelbo),
            ('gen/elbo', -nelbo),
            ('gen/kl_z', kl),
            ('gen/rec', rec),
        ))

        return loss, summaries

    def sample_sigmoid(self, batch):
        z = self.sample_z(batch)
```

```
        return self.compute_sigmoid_given(z)

    def compute_sigmoid_given(self, z):
        logits = self.dec.decode(z)
        return torch.sigmoid(logits)

    def sample_z(self, batch):
        return ut.sample_gaussian(
            self.z_prior[0].expand(batch, self.z_dim),
            self.z_prior[1].expand(batch, self.z_dim))

    def sample_x(self, batch):
        z = self.sample_z(batch)
        return self.sample_x_given(z)

    def sample_x_given(self, z):
        return torch.bernoulli(self.compute_sigmoid_given(z))
```

Claude:

I'll implement the negative_elbo_bound method for the VAE.
The ELBO (Evidence Lower Bound) for a VAE is:
ELBO = E_q(z|x)[log p(x|z)] - KL(q(z|x) || p(z))
So the negative ELBO is:
nelbo = KL(q(z|x) || p(z)) - E_q(z|x)[log p(x|z)]
Which can be decomposed as:

- kl: KL divergence between the encoder distribution q(z|x) and the prior p(z)
- rec: Negative reconstruction log-likelihood (negative of log p(x|z))

Here's the implementation:
python

```
def negative_elbo_bound(self, x):
    """
    Computes the Evidence Lower Bound, KL and, Reconstruction costs

    Args:
        x: tensor: (batch, dim): Observations

    Returns:
        nelbo: tensor: (): Negative evidence lower bound
        kl: tensor: (): ELBO KL divergence to prior
        rec: tensor: (): ELBO Reconstruction term
    """
    mean, var = self.enc.encode(x)
```

```
    z = ut.sample_gaussian(mean, var)
    logits = self.dec.decode(z)


################################################################################
####
    # TODO: Modify/complete the code here
    # Compute negative Evidence Lower Bound and its KL and Rec decomposition
    #
    # Note that nelbo = kl + rec
    #
    # Outputs should all be scalar

################################################################################
####

    # KL divergence: KL(q(z|x) || p(z))
    # q(z|x) = N(mean, var), p(z) = N(0, I)
    pm = self.z_prior_m.expand(mean.shape)  # prior mean (zeros)
    pv = self.z_prior_v.expand(var.shape)   # prior variance (ones)
    kl = ut.kl_normal(mean, var, pm, pv)    # (batch,)
    kl = kl.mean()  # average over batch

    # Reconstruction term: -E_q(z|x)[log p(x|z)]
    # p(x|z) is Bernoulli, so we use log_bernoulli_with_logits
    log_prob = ut.log_bernoulli_with_logits(x, logits)  # (batch,)
    rec = -log_prob.mean()  # negative log-likelihood, averaged over batch

    # Negative ELBO
    nelbo = kl + rec


################################################################################
####
    # End of code modification

################################################################################
####
    return nelbo, kl, rec
```

Explanation:

1. KL term: Uses ut.kl_normal to compute KL divergence between:
    - q(z|x) = N(mean, var) — the encoder's approximate posterior
    - p(z) = N(0, I) — the prior (defined by self.z_prior_m and self.z_prior_v)

2. Reconstruction term: Uses ut.log_bernoulli_with_logits to compute log p(x|z) where x is modeled as Bernoulli (appropriate for binarized MNIST). The negative of this gives us the reconstruction loss.
3. Averaging: Both terms are averaged over the batch to produce scalar outputs.
4. nelbo = kl + rec: The negative ELBO is the sum of the KL divergence and negative reconstruction log-likelihood.

User:

MAML for Classification

Suppose we want to learn an effective set of feature weights for performing classification tasks. We will use the same setup as the regression problem, but now our training data consists of pairs (x_i, z_i), where $z_i = \text{sgn}(f(x_i))$. The function f represents the underlying true function that is sampled from a distribution D_T as before.

Given a set of feature weights and their corresponding weighted features ϕ_w, we solve the logistic regression problem to learn a set of coefficients α. Using these coefficients, we assign a label to a test point x_{test} as $\hat{z}_{test} = sgn(\langle \alpha, \phi_w(x_{test}) \rangle)$. The test loss of interest is the classification error $\mathbb{E}[\hat{z}_{test} \neq z_{test}]$, where the expectation is taken over the randomness in the test point. Since the test loss is not differentiable, we use the logistic loss during training (logistic regression).

Next, we describe the process of learning the coefficients $\alpha \in \mathbb{R}^d$. We have training data pairs (x_i, z_i), where x_i represents a data point, and z_i denotes its label from the set ${+1, -1}$. For each point x_i, we have a set of weighted features $\phi_w(x_i)$.

Recall that a standard logistic function is defined as $$g(t) = \frac{1}{e^{-t}+1}$$ (refer to: https://en.wikipedia.org/wiki/Logistic_function), which possesses the useful property of $g(-t) = 1 - g(t)$. Consequently, we can use it to model the binary probability: given a value t that might belong to two classes with labels $+1$ and -1, we can model the probability of t being part of class $+1$ as $p_1(t) = g(t)$, and conveniently obtain $1 - g(t) = p_{-1}(t)$. In our classification problem setup, this $t = z_i\langle \alpha, \phi_w(x_i) \rangle$. Thus, $g(t)$ can be interpreted as the probability of "correct classification", because for a correct prediction, $t = z_i\langle \alpha, x_i \rangle$ will always be positive, and its probability $g(t)$ will be appropriately limited to 1.

Therefore, to maximize the total log probability of predicting the correct label on all datapoints we find the α that maximizes
$$\Sigma_i \log{ \frac{1}{e^{-z_i\langle \alpha, \phi_w(x_i) \rangle}+1} }.$$ Flipping the sign, this is equivalent to finding the α that minimizes the loss

$$ - \Sigma_i \log{ \frac{1}{e^{-z_i\langle \alpha, \phi_w(x_i) \rangle}+1} },$$ as listed here: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

One way to find this α is by using gradient descent similar to what we did in the regression setting. In the regression setting we used the squared loss but in the classification setting we use the logistic loss.

Complete the following code and make sure it run without any errors though the meta learning is being done on regression tasks and test losses are being computed on the regression tasks.

```
def get_post_data_classification(x_type, phi_type, k_idx, num_tasks_test, d,
n_train_post_range, n_test, noise_prob):

  k_val_test = np.random.uniform(-1, 1, size=(len(k_idx), num_tasks_test))
  k_val_test /= np.linalg.norm(k_val_test, axis=0)

  data_dict = {'train': {}, 'test': {}}

  # features_post_complete = featurize(x_train_post_complete, phi_type=phi_type,d=d,
normalize = True)
  x_test = generate_x(n_test, 'uniform_random')
  features_test = featurize(x_test, phi_type=phi_type, d=d, normalize=True)
  data_dict['test']['features'] = features_test

  data_dict['test']['x'] = x_test
  data_dict['test']['y'] = []
  for i in range(num_tasks_test):
     k_val_post = k_val_test[:, i]
     y_test = generate_y(features_test, k_idx, k_val_post)
     data_dict['test']['y'].append(y_test)
  for n_train_post in n_train_post_range:
     data_dict['train'][n_train_post] = {}

     x_train_post = generate_x(n_train_post, x_type)
     features_post = featurize(x_train_post, phi_type=phi_type, d=d, normalize=True)

     data_dict['train'][n_train_post]['x'] = x_train_post
     data_dict['train'][n_train_post]['features'] = features_post
     data_dict['train'][n_train_post]['y'] = []

     for i in range(num_tasks_test):
        k_val_post = k_val_test[:, i]
        y_post = generate_y(features_post, k_idx, k_val_post)
        y_post = add_label_noise(y_post, noise_prob)
```

```
        # y_post += np.random.normal(0, noise_std, y_post.shape)
        data_dict['train'][n_train_post]['y'].append(y_post)
    return data_dict

def meta_update_classification(w_t, feature_weights_t, n_train_meta, phi_type, d, k_idx, k_val,
noise_prob):
    x_meta = generate_x(n_train_meta, 'uniform_random')
    features_meta = featurize(x_meta, phi_type=phi_type, d=d, normalize=True)
    features_meta_t = torch.tensor(features_meta)
    y_meta = generate_y(features_meta, k_idx, k_val)
    y_meta = add_label_noise(y_meta, noise_prob)

    y_meta_t = torch.tensor(y_meta)
    y_meta_pred_t = features_meta_t @ w_t


##############################################################################
    # TODO: Complete the following code

##############################################################################
    z_meta_t = my_sign(y_meta_t)  # Convert to labels {-1, +1}
    # Logistic loss: sum of log(1 + exp(-z * y_pred))
    loss = torch.sum(torch.log(1 + torch.exp(-z_meta_t * y_meta_pred_t)))

##############################################################################

    return loss


def meta_learning_classification(params_dict):
    seed = params_dict["seed"]
    n_train_inner = params_dict["n_train_inner"]
    n_train_meta = params_dict["n_train_meta"]
    # n_train_post = params_dict["n_train_post"]
    n_test_post = params_dict["n_test_post"]
    x_type = params_dict["x_type"]
    d = params_dict["d"]
    phi_type = params_dict["phi_type"]
    k_idx = params_dict["k_idx"]
    optimizer_type = params_dict["optimizer_type"]
    stepsize_meta = params_dict["stepsize_meta"]
    num_inner_tasks = params_dict["num_inner_tasks"]
    num_tasks_test = params_dict["num_tasks_test"]
    num_stats = params_dict["num_stats"]
```

```
  num_iterations = params_dict["num_iterations"]
  noise_prob = params_dict['noise_prob']
  num_n_train_post_range = params_dict['num_n_train_post_range']

  stepsize_inner = params_dict["stepsize_inner"]
  num_gd_steps = params_dict['num_gd_steps']

  # Set seed:
  np.random.seed(seed)
  torch.manual_seed(seed)

  # Parameters
  stats_every = num_iterations // num_stats
  init_n_train_post_range, init_avg_test_loss, init_top_10_loss, init_bot_10_loss = None, None,
None, None

  dm = DummyModel(d)
  # Define meta parameter optimizer
  if optimizer_type == 'SGD':
     opt_meta = torch.optim.SGD([dm.feature_weights], lr=stepsize_meta)
  elif optimizer_type == 'Adam':
     opt_meta = torch.optim.Adam([dm.feature_weights], lr=stepsize_meta)
  else:
     raise ValueError

  # Meta training loop
  # Get post train and test data
  n_train_post_range = np.logspace(np.log10(24), np.log10(3 * d),
num_n_train_post_range).astype('int')

  # must_have_points  = [n_train_inner, len(k_idx)-1, len(k_idx), len(k_idx)+1]
  must_have_points = [n_train_inner]
  for point in must_have_points:
     if point not in n_train_post_range:
        n_train_post_range = np.hstack([n_train_post_range, point])

  n_train_post_range = np.sort(n_train_post_range)
  n_train_post_range = np.unique(n_train_post_range)
  data_dict = get_post_data_classification(
     x_type, phi_type, k_idx, num_tasks_test, d, n_train_post_range, n_test_post, noise_prob)

  for i in range(num_iterations):
     opt_meta.zero_grad()
```

```
        # Get x and features
        x = generate_x(n_train_inner, x_type)
        features = featurize(x, phi_type=phi_type, d=d, normalize=True)
        features_t = to_tensor(features)

        # Loop over inner tasks
        for t in range(num_inner_tasks):
            # Get random coefficients
            k_val = np.random.uniform(-1, 1, size=len(k_idx))
            k_val /= np.linalg.norm(k_val)
            dm.coeffs = torch.nn.Parameter(torch.zeros(d).double())

            # Generate y
            y = generate_y(features, k_idx, k_val)
            # y += np.random.normal(0, noise_std, y.shape)

            y = add_label_noise(y, noise_prob)
            y_t = torch.tensor(y)

            opt_inner = torch.optim.SGD([dm.coeffs], lr=stepsize_inner, weight_decay=1e-5)
            with higher.innerloop_ctx(dm, opt_inner, copy_initial_weights=False,
track_higher_grads=True) as (mod, opt):

                for j in range(num_gd_steps):
                    y_pred = mod(features_t)

                    ###################################################################
                    # TODO: Complete the following code
                    ###################################################################
                    z_t = my_sign(y_t)  # Convert to labels {-1, +1}
                    # Logistic loss: sum of log(1 + exp(-z * y_pred))
                    loss = torch.sum(torch.log(1 + torch.exp(-z_t * y_pred)))
                    ###################################################################

                    opt_inner.zero_grad()
                    loss.backward(retain_graph=True)
                    opt.step(loss)

                # Meta update
                meta_loss = meta_update_classification(
                    mod.coeffs * mod.feature_weights, mod.feature_weights, n_train_meta, phi_type, d,
k_idx, k_val, noise_prob)
                meta_loss.backward(retain_graph=True)
```

```
    if i == 0:
        # n_train_post_range = np.logspace(0,np.log10(3*d), 40).astype('int')
        print("-" * 70)
        print("Iteration: ", i)
        # Oracle stats

        # Start comment for dev
        oracle_feature_weights = np.zeros(d)
        oracle_feature_weights[k_idx] = 1
        oracle_n_train_post_range, oracle_avg_test_loss, oracle_top_10_loss,
oracle_bot_10_loss = test_classification_oracle(
            data_dict, feature_weights=oracle_feature_weights, x_type=x_type)

        # plt.plot(oracle_n_train_post_range, oracle_avg_test_loss)
        # plt.show()

        # oracle_n_train_post_range,oracle_avg_test_loss, oracle_top_10_loss,
oracle_bot_10_loss = None, None, None, None

        zero_avg_loss, zero_top_10_loss, zero_bot_10_loss =
test_classification_zero(data_dict)
        # zero_avg_loss, zero_top_10_loss, zero_bot_10_loss  = None, None, None

        # End comment for dev

        feature_weights = to_numpy(dm.feature_weights)
        init_n_train_post_range, init_avg_test_loss, init_top_10_loss, init_bot_10_loss =
test_classification(
            data_dict, feature_weights)

        visualize_test_loss_classification(
            0, n_train_inner, init_n_train_post_range, init_avg_test_loss, init_top_10_loss,
init_bot_10_loss,
            oracle_n_train_post_range=oracle_n_train_post_range,
oracle_avg_test_loss=oracle_avg_test_loss,
            oracle_top_10_loss=oracle_top_10_loss, oracle_bot_10_loss=oracle_bot_10_loss,
zero_avg_loss=zero_avg_loss,
            zero_top_10_loss=zero_top_10_loss, zero_bot_10_loss=zero_bot_10_loss,
noise_prob=noise_prob)

        visualize_prediction_classification(
            data_dict, feature_weights, n_train_inner)

    # Stats
```

```
        if (i+1) % stats_every == 0 or i == num_iterations - 1:

            print("-" * 70)
            print("Iteration: ", i + 1)

            feature_weights = to_numpy(dm.feature_weights)

            n_train_post_range, avg_test_loss, top_10_loss, bot_10_loss = test_classification(
                data_dict, feature_weights)
            visualize_test_loss_classification(
                i, n_train_inner, n_train_post_range, avg_test_loss,
                top_10_loss, bot_10_loss, init_n_train_post_range, init_avg_test_loss,
                init_top_10_loss, init_bot_10_loss,
                oracle_n_train_post_range=oracle_n_train_post_range,
                oracle_avg_test_loss=oracle_avg_test_loss,
                oracle_top_10_loss=oracle_top_10_loss,
                oracle_bot_10_loss=oracle_bot_10_loss,
                zero_avg_loss=zero_avg_loss,
                zero_top_10_loss=zero_top_10_loss,
                zero_bot_10_loss=zero_bot_10_loss,
                noise_prob=noise_prob)
            visualize_prediction_classification(
                data_dict, feature_weights, n_train_inner)

        opt_meta.step()  # Finally update meta weights after loop through all tasks


def test_classification(data_dict, feature_weights, min_n_train_post=0):
    # plt.plot(feature_weights, 'o-')
    # plt.show()
    # print(feature_weights)
    # Added for dev
    test_loss_matrix = []

    n_train_post_range = np.sort(np.array(list(data_dict['train'].keys())))

    n_train_post_range = n_train_post_range[n_train_post_range >= min_n_train_post]

    test_data_dict = data_dict['test']
    features_test_post = test_data_dict['features']

    # n_train_post_range = np.array([32])

    for n_train_post in n_train_post_range:
```

```
        # print("n", n_train_post)
        train_data_dict = data_dict['train'][n_train_post]
        features_post = train_data_dict['features']

        test_loss_array = []
        for i in range(len(train_data_dict['y'])):

            y_post = train_data_dict['y'][i]
            # print(features_post.shape, y_post.shape)
            z_post = np.squeeze(my_sign(y_post))

            w_post, loss = solve_logistic(
                features_post, z_post, feature_weights)
            y_post_pred = features_post @ w_post

            y_test_post = test_data_dict['y'][i]
            y_test_post_pred = features_test_post@w_post

            z_post_pred = np.squeeze(my_sign(y_post_pred))
            z_test_post = np.squeeze(my_sign(y_test_post))
            z_test_post_pred = np.squeeze(my_sign(y_test_post_pred))

            # Compute the regression loss
            test_loss = np.mean(z_test_post != z_test_post_pred)
            test_loss_array.append(test_loss)

        test_loss_matrix.append(test_loss_array)

    test_loss_matrix = np.array(test_loss_matrix).T
    avg_test_loss = np.mean(test_loss_matrix, 0)

    top_10_loss = np.percentile(test_loss_matrix, 90, axis=0)
    bot_10_loss = np.percentile(test_loss_matrix, 10, axis=0)

    return n_train_post_range, avg_test_loss, top_10_loss, bot_10_loss


def test_classification_zero(data_dict):
    test_data_dict = data_dict['test']
    ys = test_data_dict['y']

    test_loss_array = []
    for i in range(len(ys)):
        y = ys[i]
```

```
        z = my_sign(y)
        z_guess = np.random.randint(low=-1, high=1, size=z.shape)
        z_guess = my_sign(z_guess)
        test_loss = np.mean(z != z_guess)
        # NOTE: for regression, the initial loss is np.mean(y**2), i.e. predicting zero for every point,
        # here the equivalent would be randomly guessing signs as labels
        test_loss_array.append(test_loss)

    avg_test_loss = np.mean(test_loss_array)
    top_10_loss = np.percentile(test_loss_array, 90)
    bot_10_loss = np.percentile(test_loss_array, 10)

    return avg_test_loss, top_10_loss, bot_10_loss


def test_classification_oracle(data_dict, x_type, feature_weights, min_n_train_post=0):
    # plt.plot(feature_weights, 'o-')
    # plt.show()
    # print(feature_weights)
    k_idx = np.where(feature_weights != 0)[0]
    feature_weights = None

    test_loss_matrix = []

    n_train_post_range = np.sort(np.array(list(data_dict['train'].keys())))

    n_train_post_range = n_train_post_range[n_train_post_range >= min_n_train_post]

    test_data_dict = data_dict['test']
    features_test_post = test_data_dict['features']

    for n_train_post in n_train_post_range:
        # print("n", n_train_post)
        train_data_dict = data_dict['train'][n_train_post]

        features_post = train_data_dict['features'][:, k_idx]
        cfeatures_test_post = features_test_post[:, k_idx]

        feature_norms = np.linalg.norm(features_post, axis=0)
        r_idx = np.where(feature_norms > 1e-6)[0]

        features_post = features_post[:, r_idx]
        cfeatures_test_post = cfeatures_test_post[:, r_idx]
```

```
    test_loss_array = []

    for i in range(len(train_data_dict['y'])):
        y_post = train_data_dict['y'][i]
        z_post = np.squeeze(my_sign(y_post))

        w_post, loss = solve_logistic(features_post, z_post, feature_weights)

        y_post_pred = features_post @ w_post

        y_test_post = test_data_dict['y'][i]
        y_test_post_pred = cfeatures_test_post@w_post

        z_test_post = np.squeeze(my_sign(y_test_post))
        z_test_post_pred = np.squeeze(my_sign(y_test_post_pred))
        test_loss = np.mean(z_test_post != z_test_post_pred)
        # print(test_loss)
        # Compute the regression loss
        test_loss_array.append(test_loss)

    test_loss_matrix.append(test_loss_array)

  test_loss_matrix = np.array(test_loss_matrix).T
  avg_test_loss = np.mean(test_loss_matrix, 0)

  top_10_loss = np.percentile(test_loss_matrix, 90, axis=0)
  bot_10_loss = np.percentile(test_loss_matrix, 10, axis=0)

  return n_train_post_range, avg_test_loss, top_10_loss, bot_10_loss
```

Claude:

Looking at the code and the explanation, I need to implement the logistic loss for classification in two places:

1. In meta_update_classification - computing the meta loss
2. In meta_learning_classification - computing the inner loop loss

Based on the markdown explanation, the logistic loss to minimize is:
−∑ilog(1e−zi⟨α,ϕw(xi)⟩+1)-\sum_i \log\left(\frac{1}{e^{-z_i\langle \alpha, \phi_w(x_i) \rangle}+1}\right)−i∑log(e−zi⟨α,ϕw(xi)⟩+11)
Which can be rewritten as:
∑ilog(1+e−zi·y^i)\sum_i \log\left(1 + e^{-z_i \cdot \hat{y}_i}\right)i∑log(1+e−zi·y^i)
where y^i=⟨α,ϕw(xi)⟩\hat{y}_i = \langle \alpha, \phi_w(x_i) \rangle y^i=⟨α,ϕw(xi)⟩ is the prediction and zi∈{−1,+1}z_i \in \{-1, +1\} zi∈{−1,+1} is the label.
Here are the two TODO completions:

1. meta_update_classification (meta loss):

python

```
def meta_update_classification(w_t, feature_weights_t, n_train_meta, phi_type, d, k_idx, k_val, noise_prob):
    x_meta = generate_x(n_train_meta, 'uniform_random')
    features_meta = featurize(x_meta, phi_type=phi_type, d=d, normalize=True)
    features_meta_t = torch.tensor(features_meta)
    y_meta = generate_y(features_meta, k_idx, k_val)
    y_meta = add_label_noise(y_meta, noise_prob)

    y_meta_t = torch.tensor(y_meta)
    y_meta_pred_t = features_meta_t @ w_t


##########################################################################
    # TODO: Complete the following code

##########################################################################
    z_meta_t = my_sign(y_meta_t)  # Convert to labels {-1, +1}
    # Logistic loss: sum of log(1 + exp(-z * y_pred))
    loss = torch.sum(torch.log(1 + torch.exp(-z_meta_t * y_meta_pred_t)))

##########################################################################

    return loss
```

2. meta_learning_classification (inner loop loss):

python

```
        for j in range(num_gd_steps):
            y_pred = mod(features_t)

            ##############################################################
            # TODO: Complete the following code
            ##############################################################
            z_t = my_sign(y_t)  # Convert to labels {-1, +1}
            # Logistic loss: sum of log(1 + exp(-z * y_pred))
            loss = torch.sum(torch.log(1 + torch.exp(-z_t * y_pred)))
            ##############################################################

            opt_inner.zero_grad()
            loss.backward(retain_graph=True)
            opt.step(loss)
```

Explanation:

1. my_sign(y) - Converts the continuous target y to binary labels z ∈ {-1, +1}
2. -z_t * y_pred - This is the negative margin. When the prediction has the same sign as the label (correct), this is negative, leading to a small loss. When they disagree (incorrect), this is positive, leading to a large loss.
3. torch.log(1 + torch.exp(...)) - This is the logistic loss (also known as softplus of the negative margin), which is a smooth, differentiable surrogate for the 0-1 classification loss.
4. torch.sum(...) - Sum over all data points (matching the formula in the problem description which uses summation).

Analysis:

Claude was able to one shot this homework. It didn't even need any clarifications and was able to also give explanations for its code afterwards. I'm pretty happy with the choice of variable names and the overall cleanliness of the code. I'm not too surprised because I think this homework in particular was quite simple relatively. I have attached the results of the VAE. and MAML